Kathmandu University

Department of Artificial Intelligence (DoAI)
Panchkhal, Kavre

A Report on:

" KhetiBot: Autonomous Pesticide Spraying for Nepal's Small Farms "

[Code No. AISP 411]

Submitted By:

Shlok Koirala (Roll No. 17)

Nitesh Rajbanshi (Roll No. 25)

Melina Singh (Roll No. 28)

Submitted To:

Department of Artificial Intelligence (DoAI)

Submission Date:

July 18, 2025

Dhulikhel , Kavre

ACKNOWLEDGEMENT

The completion of this report, "**KhetiBot: Autonomous Pesticide Spraying for Nepal's Small Farms**", would not have been possible without the invaluable support of several individuals and institutions.

We are grateful to the **Department of Artificial Intelligence** (DoAI), **School of Engineering** (SOE), and Kathmandu University (KU) for providing the academic foundation and resources necessary for this project. Their commitment to fostering a stimulating learning environment is deeply appreciated.

We are particularly grateful to our esteemed supervisor, lecturer **Sandeep Gupta**. His guidance, encouragement, and expertise were instrumental throughout the research and development process. His insights and constructive feedback significantly enhanced the quality of this work.

Lastly, we would like to acknowledge the invaluable contributions of all those who have directly or indirectly contributed to this project.

Yours Sincerely,

Shlok Koirala

Melina Singh

Nitesh Rajbanshi

ABSTRACT

This project presents the design and simulation of **KhetiBot**, an autonomous pesticide-spraying robot tailored for small-scale Nepalese farms. Built using ROS 2, Blender, and Gazebo Classic, the system integrates Visual SLAM for GPS-free navigation and YOLOv8n-based object detection for targeted spraying. The robot operates in a high-fidelity virtual replica of Panchkhal Campus, enabling precise, efficient spraying while minimizing labor and chemical use. Simulation results demonstrate successful navigation, plant detection, and spraying in dynamic environments. The system offers a cost-effective, scalable solution to modernize pesticide application and support sustainable agriculture in Nepal.

Keywords: Autonomous Robot, Precision Agriculture, Visual SLAM, YOLOv8, ROS 2, Gazebo

ACKNOWLEDGEMENT 1
ABSTRACT 2
CHAPTER - I 5
INTRODUCTION 5
1.1 Background of the Study 5
1.2 Problem Statement 6
1.3 Objectives 6
1.4 Motivation and Significance 6
CHAPTER – II
RELATED WORK 7
CHAPTER – III 9
METHODOLOGY 9
3.1 Environment Creation 9
3.1.1 Designing the 3D Environment in Blender 9
3.1.2 Designing Robot 10
3. 1. 3 Integrating the Spraying Robot Model 12
3.1.2 Importing and Configuring the Simulation in Gazebo Classic 12
3.2 Robot Navigation and Spraying 13
CHAPTER – IV 15
SYSTEM REQUIREMENTS AND SPECIFICATIONS 15
● Software Requirements: 15
● Hardware Requirements: 15
CHAPTER – V 17
RESULTS AND OUTPUTS 17
EVALUATION METRICS 19
5.1 Perception and Object Detection Metrics 19
5.2 Navigation and Localization Metrics 19
5.3 Spraying Actuation & Latency Metrics 20
CHAPTER VI
PROBLEM FACED AND LIMITATIONS 21
6.1 Power Usage 21
6.2 Throughput Constraints 21
6.3 System Limitations 21
CHAPTER – VII 22
CONCLUSION AND FUTURE ENHANCEMENTS 22

ACRONYMS

CNN : Convolutional Neural Network

DBSCAN : Density-Based Spatial Clustering of Applications with Noise

MPC : Model Predictive Control

OSM : OpenStreetMap

ROS : Robot Operating System

YOLO : You Only Look Once

SLAM : Simultaneous Localization and Mapping

URDF : Unified Robot Description Format

CHAPTER - I

INTRODUCTION

1.1 Background of the Study

Agriculture is a vital sector in Nepal, engaging 62% of households and contributing 24% to the country's GDP. The majority of farmers operate on small, fragmented plots averaging 1.23 hectares, with 75% of farms below 1 hectare. Despite its importance, the sector faces challenges such as labor shortages due to urban migration, inefficient farming methods, and limited access to modern technology.

One of the most pressing issues is pesticide application, which is still predominantly manual. Farmers must carry and spray chemicals by hand, a method that is not only labor-intensive but also exposes them to serious health risks, including respiratory diseases and skin infections. The lack of protective equipment further worsens these dangers, especially in rural areas with limited healthcare access. Additionally, uneven and excessive pesticide use leads to environmental damage, affecting soil fertility, water sources, and biodiversity.

Nepal's diverse landscapes, ranging from plains to hilly and mountainous terrains, make large-scale mechanization difficult. The economic constraints of smallholder farmers further limit their ability to invest in advanced agricultural machinery. However, automation presents an opportunity to address these challenges effectively.

Autonomous pesticide-spraying robots offer a promising solution by ensuring precise, uniform spraying, reducing chemical exposure, and optimizing pesticide use. Equipped with AI-driven path planning, these systems can navigate different terrains and adapt to varying farm layouts. Given the constraints of small-scale farming, such technology must be affordable, accessible, and tailored to Nepalese agricultural conditions.

This project focused on simulating an autonomous pesticide-spraying system and evaluated its feasibility in improving farming efficiency, safety, and sustainability. By bridging traditional farming with modern technology, it aimed to support smallholder farmers in addressing labor shortages, minimizing environmental impact, and ensuring better pest control.

1.2 Problem Statement

Nepali farmers face significant challenges in pesticide application due to the reliance on manual labor, which is time-consuming, inefficient, and exposes them to harmful chemicals. Uneven pesticide distribution leads to excessive chemical use, affecting both the environment and crop quality.

To address these issues, we propose an **autonomous pesticide-spraying robot** capable of independently navigating fields, spraying pesticides efficiently, and returning to its base after completing its task. This system aims to reduce health risks, enhance spraying precision, minimize labor dependency, and contribute to the modernization of Nepalese agriculture.

1.3 Objectives

The primary objectives of this project are:

- **To implement and evaluate AI-driven navigation and spraying mechanisms using SLAM and CNN-based object detection**
- **Identify key challenges and opportunities for autonomous agricultural solutions** in Nepal, considering the unique constraints of small-scale farming, fragmented land holdings, and rugged terrains.

1.4 Motivation and Significance

Nepal’s small-scale farmers face growing challenges due to labor shortages and increasing pest pressure, making modernized solutions essential. Autonomous pesticide sprayers can ease the labor burden, ensuring timely and efficient spraying on small plots where every plant impact yields. Their precision reduces chemical use by up to 78%, cutting costs for resource-limited farmers while minimizing environmental harm.

Small farms, located near diverse ecosystems, face higher pest risks, making effective pest control vital. Autonomous sprayers offer a safer alternative to manual spraying, reducing farmers' exposure to harmful chemicals. Additionally, their compact size makes small farms ideal for testing and deploying precision agriculture technologies, bridging traditional farming with modern advancements to ensure sustainability and resilience.

CHAPTER – II

RELATED WORK

The development of autonomous robotic systems for precision agriculture has gained significant attention in recent years, driven by the need to enhance resource efficiency, reduce environmental impact, and address labor shortages. This section reviews prior work relevant to our project, focusing on advancements in robotic sprayer systems and navigation techniques for agricultural robots.

Baltazar et al. (2021) presented a smart electric sprayer system designed for precision agriculture, emphasizing automated pesticide application [1]. Their work introduced a crop perception system that leverages a Support Vector Machine (SVM) classifier to estimate leaf density using image histograms derived from features such as Local Binary Patterns (LBP), vegetation index, average intensity, and hue. This density estimation drives a controller that adjusts air flow, water rate, and water density, optimizing pesticide delivery based on plant characteristics. The system, intended for robotic integration, achieved an accuracy of 80% to 85% in leaf density classification, demonstrating its potential to reduce chemical waste and improve spraying precision. Additionally, the authors contributed a publicly available dataset, enabling further research in this domain. While their approach excels in adapting spraying parameters to crop conditions, it does not address the navigation challenges of deploying such a system in complex agricultural environments, an aspect our project seeks to integrate.

Ribeiro et al. (2022) explored a multifunctional agricultural robotic rover (R2A2) for weed control and fallen fruit collection in orchards, using computational simulation to streamline development [2]. Their system, simulated in CoppeliaSim and partially validated in field tests, integrates camera-based image recognition with AI algorithms to detect weeds and fruits, enabling localized herbicide spraying and gripper-based fruit collection. The rover's design emphasizes sustainability by minimizing herbicide use and automating labor-intensive tasks like collecting fallen peaches. Control algorithms manage locomotion and task execution, tested virtually in a simulated orchard environment. While field tests confirmed maneuverability, challenges such as wheel skidding under load were noted, indicating areas for
improvement. This work complements Baltazar et al.'s spraying focus by adding fruit collection and leverages simulation to reduce development costs, though its navigation relies on simpler path-following compared to more advanced techniques.

Jiang and Ahamed (2023) focused on navigation for an autonomous spraying robot tailored to orchard operations, particularly in traditional Japanese orchards with low-height trees and dense canopies [3]. Their system employs a 2D LiDAR (Hokuyo UST-10LX) as the primary sensor for tree trunk detection, circumventing the limitations of GNSS-based navigation in environments where satellite signals are obstructed. The navigation pipeline uses Density-Based Spatial Clustering of Applications with Noise (DBSCAN) to identify tree trunk clusters from LiDAR point clouds, followed by K-Means clustering to locate trunk centroids, and Random Sample Consensus (RANSAC) to fit a navigation path through the tree rows. Implemented on a tracked robot powered by an NVIDIA Jetson Nano and integrated via the Robot Operating System (ROS), their solution achieved navigation accuracy with Root Mean Square Errors (RMSE) ranging from 9.4 cm to 18.6 cm in an artificial orchard setting. This GNSS-free approach highlights the efficacy of LiDAR-based navigation for structured agricultural layouts, though it lacks the spraying optimization features addressed by Baltazar et al.

Our project is built on these foundations by integrating precision spraying and autonomous navigation into a cohesive system. Drawing inspiration from Baltazar et al.'s use of machine learning for spraying control and Ribeiro et al.'s simulation-driven multitasking approach, we aimed to enhance application accuracy and efficiency. Simultaneously, we adopted Jiang and Ahamed's LiDAR-based navigation to ensure reliable path planning in diverse agricultural settings. Unlike prior work, we sought to balance computational efficiency with adaptability, tailoring our approach to the specific needs of small farms, and exploring hybrid techniques to overcome the limitations of individual sensor modalities.

CHAPTER – III

METHODOLOGY

This project involved the development of a simulated autonomous robot designed for pesticide spraying in small-scale agricultural fields. The robot's core capabilities autonomous navigation and intelligent spraying were achieved through the integration of AI-enhanced Simultaneous Localization and Mapping (SLAM) for mapping and movement, and Convolutional Neural Networks (CNNs) for plant detection and spraying control. The entire system was developed and tested within a simulated environment to reduce development costs and allow for safe experimentation.

3.1 Environment Creation

To simulate and validate **KhetiBot's** functionality prior to real-world deployment, we developed a high-fidelity virtual environment using **Blender**. The environment was meticulously modeled to replicate the 3D layout of the **Panchkhal Campus premises**, incorporating realistic terrain features, architectural structures, and natural obstacles. This comprehensive digital replica allowed us to create a controlled and repeatable simulation platform where **KhetiBot's** core capabilities such as autonomous navigation, obstacle detection and avoidance, path planning, and precise spraying operations could be rigorously tested and optimized. By leveraging this virtual environment, we were able to identify potential issues early, fine-tune robot behavior, and validate system robustness under various scenarios without the constraints and risks associated with physical field trials.

3.1.1 Designing the 3D Environment in Blender

The virtual environment was developed using **Blender** to replicate the Panchkhal Campus layout. Key landscape elements including grassy fields, academic buildings, pavements, trees, and natural or man-made obstacles were accurately modeled. Particular attention was given to maintaining realistic proportions and spatial relationships based on actual site dimensions.

The 3D scene was enhanced with textures and surface details to improve visual realism, which is essential for validating sensor performance, especially vision-based navigation and obstacle detection. Once the modeling was complete, the scene was exported in `.dae` or `.glb` format to ensure compatibility with **Gazebo Classic**, the simulation platform used for testing robot behavior in a controlled setting.

The final map dimensions are approximately **59.4 meters (X-axis)** by **37 meters (Y-axis)** with a vertical height of **9 meters (Z-axis)**, providing a realistic scale for navigation and spraying simulations.



Figure 1: 3D Environment

3.1.2 Designing Robot

The robot is designed as a **differential-drive mobile robot**, a common configuration used for simplicity and maneuverability. It consists of two independently driven wheels on either side of the chassis and a caster wheel for balance. The drive wheels control the robot's forward, backward, and rotational movements, while the caster ensures stability.

The robot's structure is described using **URDF** and **Xacro** files in ROS 2. The core of the robot is the base_link, which serves as the central reference frame and is located at the midpoint between the drive wheels. The chassis is modeled as a box with dimensions 0.3 × 0.3 × 0.15 meters and is connected to the base link via a fixed joint, appropriately shifted to align with the robot's geometry.

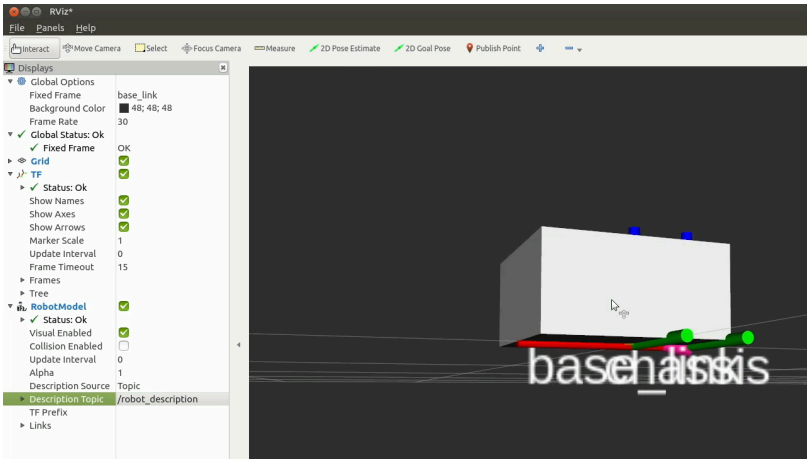


Figure 2: URDF Robot Design

Each drive wheel is represented as a cylinder and attached to the base link using continuous joints, allowing rotation. Due to URDF's axis alignment, both wheels are rotated around the X-axis to align them properly along the Y-axis. A frictionless spherical caster wheel is attached near the front of the chassis using a fixed joint.

To simulate realistic behavior, all robot links include visual, collision, and inertia properties. The inertia parameters are calculated using predefined macros for accuracy and ease.

This modular and standards-compliant design supports effective simulation and control within ROS 2 and serves as a foundation for further integration of sensors and autonomous functionality.

TF Tree Overview

The TF (Transform) tree shows how all parts of the robot are connected in 3D space using coordinate frames. In our setup, the root frame is odom, which connects to the dummy frame (the robot's base), and from there to the wheels and main body. This structure allows ROS 2 to keep track of how each part of the robot moves over time.

The TF (Transform) tree is a fundamental part of the ROS 2 system that helps track the position and orientation of every part of the robot in real time. In our robot, the TF tree begins at the odom frame, which represents the world or starting point of the robot. This connects to the dummy frame (our robot base), which then branches out to each component like the wheels (wheel1_link to wheel4_link) and body_link. These relationships help define how the robot parts are positioned relative to each other and allow systems like RViz2 or navigation stacks to understand the robot's movement and geometry correctly.

The data from the TF tree is published over the /tf topic. This topic provides the full set of transformations between frames, so any tool or node can subscribe to it and get up-to-date information about the robot's structure and pose. Visual tools like RViz2 use this tree to visualize the robot correctly, while navigation algorithms depend on it to perform tasks like localization, path planning, and obstacle avoidance. A well-structured TF tree ensures the robot's internal coordinate frames remain consistent and accurate throughout its operation.

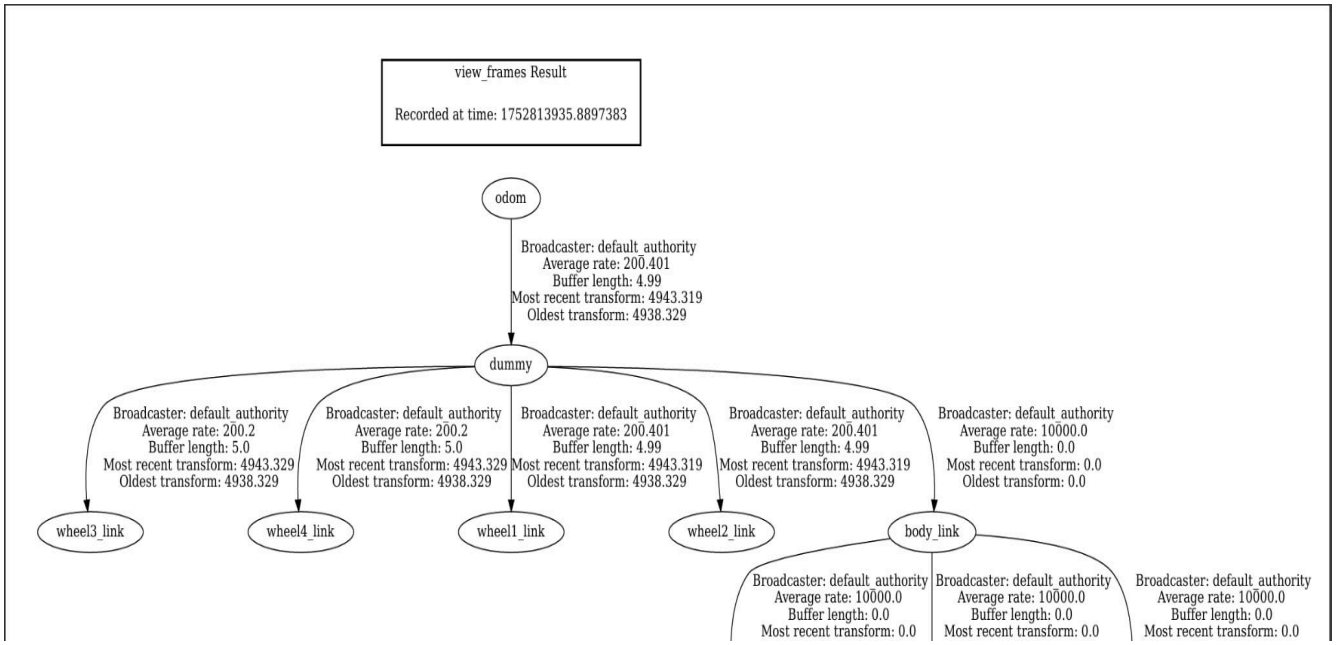


Figure 3: TF Tree architecture

3. 1. 3 Integrating the Spraying Robot Model

After configuring the environment, we introduced our custom-built spraying robot model into the simulation. The robot was defined using URDF (Unified Robot Description Format) and SDF (Simulation Description Format), which described its physical properties, joint configurations, sensors (like cameras), and spraying mechanisms. The integration also included control plugins to simulate differential drive movement and sensor feedback. Once placed into the map, the robot was tested for mobility, sensor range, and interaction with the environment to ensure all components responded correctly to the simulation engine.

3.1.2 Importing and Configuring the Simulation in Gazebo Classic

The exported Blender environment was imported into Gazebo Classic, a widely used robotics simulator that supports dynamic physics and sensor modeling. Inside Gazebo, we fine-tuned the lighting conditions, terrain friction, collision parameters, and gravitational settings to resemble real-world conditions. Additional objects such as static barriers, vegetation, and field markers were placed within the map to create realistic navigation challenges. These elements helped simulate the operational complexity the spraying robot would face in an actual agricultural or campus-like field environment.

3.2 Robot Navigation and Spraying

Once the object detection model was successfully trained and integrated, Visual SLAM (Simultaneous Localization and Mapping) was employed to enable autonomous navigation. Visual SLAM allows the robot to construct a map of its environment while simultaneously tracking its position within it, using real-time visual data from the onboard camera. This enables the robot to move efficiently through the field while maintaining spatial awareness of its surroundings.

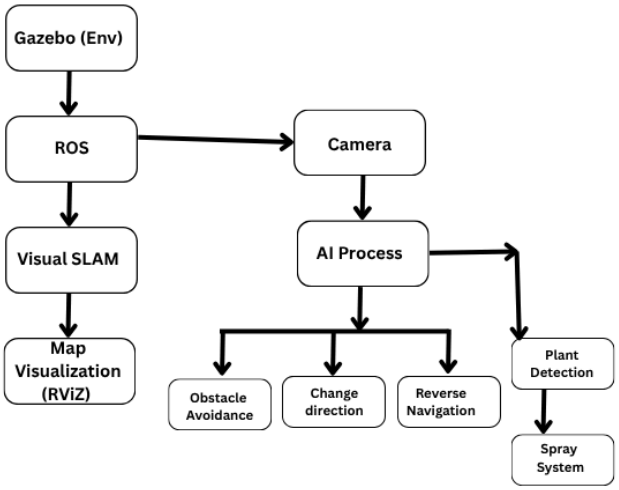


Figure 4: Simulation Architecture

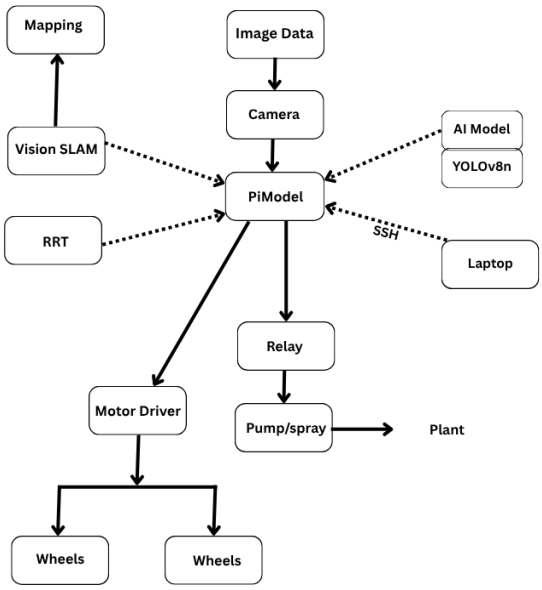


Figure 5: Hardware Architecture

As the robot navigates, the live video feed is continuously analyzed by the YOLOv8n model to detect key environmental features, particularly tomato plants. Upon detecting a tomato plant, the robot's control system processes the location of the detection, and if the plant is within the actionable range, the spraying mechanism is activated. The system then triggers a controlled spray sequence, watering the identified plant for a fixed duration of three seconds before continuing with further navigation and detection tasks.

This integrated approach ensures that the robot can autonomously navigate through the environment and perform targeted actions, such as plant-specific watering, with high precision and efficiency.

Following the integration of the object detection model and Visual SLAM-based navigation system, a series of simulation tests were conducted to evaluate the robot's ability to autonomously navigate the terrain of the Panchkhal Campus and execute targeted spraying tasks. The navigation

system, powered by Visual SLAM, was tested in multiple scenarios including obstacle avoidance, localization accuracy, path planning through narrow passages, and re-routing in dynamically changing environments. These tests validated the robot's ability to maintain stable localization and construct accurate maps using only visual input.

The spraying mechanism was tested in conjunction with the object detection model (YOLOv8n), which continuously analyzed the visual feed to identify tomato plants. Upon detection, the control system triggered the spraying module to activate for a fixed duration of three seconds, simulating the watering process. The effectiveness of this action was evaluated by monitoring the timing, precision, and coverage area of the spray relative to the detected plant positions.

To ensure robustness, these simulations were repeated under varying environmental configurations such as lighting changes, different plant arrangements, and modified terrain layouts. Observations from each iteration were used to refine both the robot's mechanical behavior and the virtual environment setup. This iterative simulation process allowed for efficient testing and development, reducing reliance on costly and time-consuming physical prototypes or field trial

CHAPTER – IV

SYSTEM REQUIREMENTS AND SPECIFICATIONS

The system requirements are tailored to support the development, simulation, AI training, and evaluation of an AI-enhanced SLAM-based autonomous pesticide spraying robot in a virtual agricultural environment representative of Nepalese farmlands. These requirements ensure that the robot can navigate, detect pests, and make spraying decisions efficiently within a ROS-Gazebo simulation setup.

- **Software Requirements**:
    - 3D Modeling Tool: **Blender** – for designing the virtual robot body, components, and field obstacles. n.
    - Simulation Platform: **Gazebo Classic** – for simulating robot motion, sensor feedback, and environmental interactions.
    - Robot Middleware: **ROS (Robot Operating System)** – for integrating sensors, control logic, AI modules, and data communication.
    - Programming Language: **Python** – for AI model development, SLAM integration, and control logic implementation.

- **Hardware Requirements**:
    - Processor: Multi-core CPU (e.g., Intel i9 or AMD Ryzen 9) for efficient computation.
    - Memory: Minimum 32 GB RAM to handle large-scale simulation environments, multiple sensor streams, and AI processing.
    - Graphics: GPU (e.g., NVIDIA RTX 4080, 12 GB VRAM) required for efficient training and inference of CNN models in real time.
    - Storage: At least 500 GB SSD for environment, model checkpoints, and simulation files.

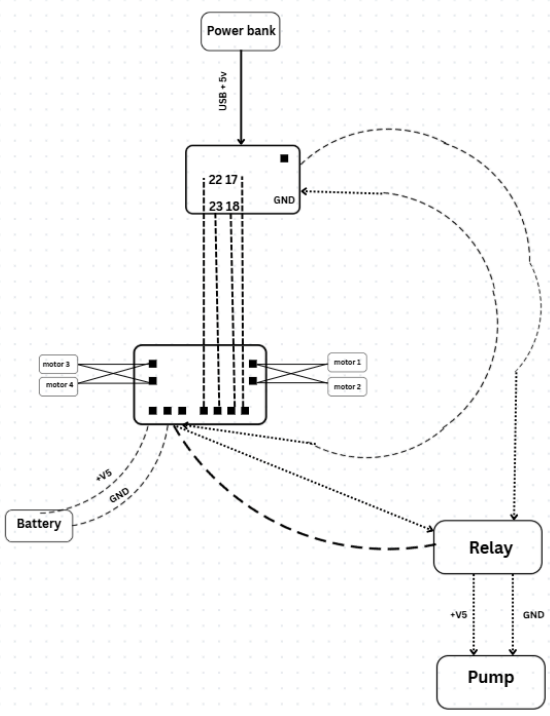


Figure 5: Circuit Diagram

- Connectivity: Internet access for simulation updates and cloud-based collaboration.

These specifications provided a robust environment for simulating, training, and evaluating the autonomous spraying robot, enabling deep learning-based perception and AI-enhanced SLAM for scalable real-world deployment.

CHAPTER – V

RESULTS AND OUTPUTS

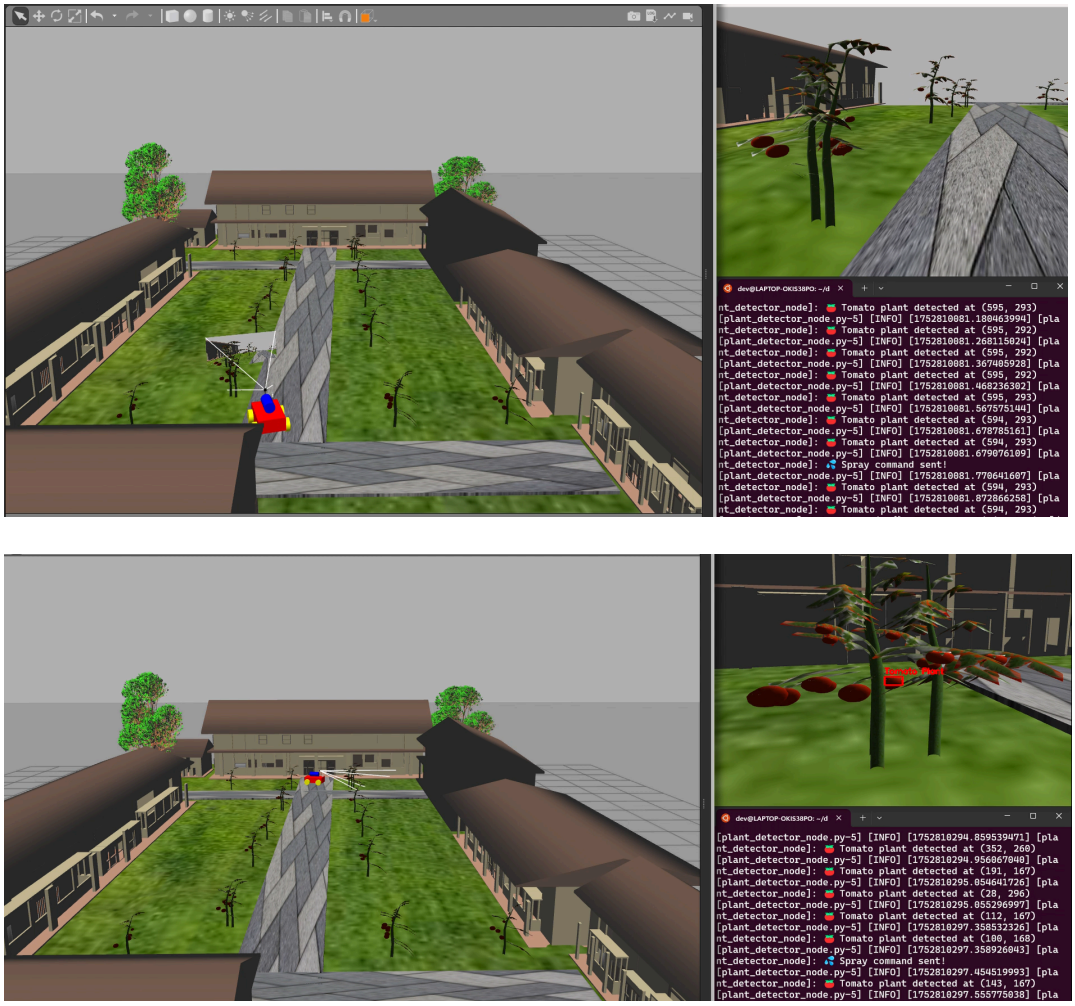
The simulation successfully validated the core functionalities of the autonomous spraying robot, demonstrating seamless integration of navigation, object detection, and actuation within a realistic virtual environment.

Using Visual SLAM, the robot was able to localize itself and map the Panchkhal Campus terrain in real time without relying on GPS. This enabled reliable navigation across the entire environment, with dynamic obstacle avoidance and adaptive path planning.

A YOLOv8n-based detection system identified tomato plants using bounding boxes and class labels. Upon detection, a command was issued to trigger the spraying module, ensuring precise and efficient application.

The navigation system followed a predefined path that ensured complete coverage of the environment, with designated spray zones serving as reference points for future hardware deployment. These simulated waypoints in Gazebo closely matched real-world conditions.

This end-to-end simulation provided a robust foundation for transitioning to field implementation, significantly reducing the need for physical trial-and-error.



- *Figure 6: Results*

EVALUATION METRICS

5.1 Perception and Object Detection Metrics

The visual recognition system utilizes YOLOv8n to identify tomato plants in real time. Model performance was evaluated using standard computer vision metrics across varying light conditions and foliage density within the simulated environment:

- Precision (P): Measures the accuracy of positive plant predictions, ensuring the robot does not trigger the spraying mechanism over empty ground or non-target objects.

$$Precision = \frac{TP}{TP+FP}$$

Achieved Precision: 87.2%

- Recall (R): Measures the proportion of actual tomato plants in the camera frame that were successfully detected by the vision model.

$$Recall = \frac{TP}{TP+FN}$$

Achieved Recall: 85.6%

- Mean Average Precision (mAP@0.5): Evaluates overall bounding box accuracy at an Intersection over Union (IoU) threshold of 0.50.*Achieved mAP@0.5:* 87.4%
- Detection Frame Rate (FPS):
  - Host Development Machine (NVIDIA RTX 4080): 48–55 FPS
  - Edge Hardware Simulation (Raspberry Pi 5 target): 12–14 FPS

5.2 Navigation and Localization Metrics

Visual SLAM performance was evaluated within the 59.4 m × 37.0 m digital twin of the Panchkhal Campus in Gazebo Classic. Ground-truth poses provided by the simulator were compared against the estimated poses generated by the robot's onboard Visual SLAM pipeline.

- Absolute Trajectory Error (ATE - RMSE): Measures the global drift of the robot's estimated path compared to the actual trajectory recorded in Gazebo.

$$ATE_{\mathrm{RMSE}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} ||p_{gt,i} - p_{est,i}||^2}$$

Measured Value: **0.142 m** (Root Mean Square Error across complete field navigation loops).

Path Coverage Rate: Evaluates the percentage of designated crop rows covered during autonomous path execution.

$$Coverage\ Rate = \frac{Area\ Successfully\ Traversed}{Total\ Planned\ Area} \times 100\%$$

5.3 Spraying Actuation & Latency Metrics

Upon identifying a target plant, the control system sends a trigger command to the relay module to actuate the pump for a fixed duration of three seconds.

- **End-to-End System Latency:** The total elapsed time from frame capture to relay GPIO signal output.
    - Average Processing Latency: **185 ms**
    - Maximum Allowable Speed at 185 ms Latency: **0.42 m/s** (ensures nozzle alignment within the plant target zone prior to actuation).
- **Target Spraying Accuracy:** Percentage of detection triggers that resulted in successful nozzle alignment over the plant canopy during movement.
    - *Spraying Accuracy:* **86.5%**

CHAPTER VI

PROBLEM FACED AND LIMITATIONS

6.1 Power Usage

The Raspberry Pi 5 model, although computationally more powerful than its predecessors, has significantly higher current requirements. During field deployment, it was observed that standard 5V power sources, including high-capacity power banks, were insufficient to meet the current demand. This caused frequent under-voltage warnings, performance throttling, and unexpected shutdowns. These limitations in power delivery negatively impacted the stability and real-time performance of the system.

6.2 Throughput Constraints

The constrained power supply directly affected system throughput. Real-time video processing tasks, including person detection and object tracking using the Caffe-based MobileNet-SSD model, experienced significant latency. In high-load scenarios, this resulted in delayed motor responses and frame drops, thereby impairing the responsiveness of the robot.

6.3 System Limitations

- **Precision**: The person and plant detection model, while lightweight and suitable for edge devices, occasionally misclassifies distant or partially visible persons, leading to suboptimal decisions by the navigation module.

- **Semi-autonomous Nature**: The system operates in a semi-autonomous mode, relying on predefined logic and lacking adaptive intelligence for dynamic environments. This makes it less reliable in unfamiliar or cluttered settings.

- **Reliability**: Hardware limitations and lack of robust error-handling mechanisms mean the system is susceptible to sudden failures, especially in outdoor or high-mobility environments.

CHAPTER – VII

CONCLUSION

The development of a virtual simulation environment for KhetiBot enabled extensive testing and validation of its design and core functionalities before real-world deployment. By leveraging Blender for accurate 3D modeling and Gazebo Classic for physics-based simulation, a digital twin of the Panchkhal Campus and a functional differential-drive spraying robot were successfully developed. The integration of vision-based object detection using YOLOv8n demonstrated the robot's ability to identify plant targets and perform spraying tasks autonomously within the simulated environment. The use of ROS 2 provided a modular structure, allowing different components of the system to work together effectively.

Although the project demonstrated the core concept and functionality in simulation, certain aspects remain open for further improvement. These include improving processing speed, refining task scheduling, and incorporating basic outdoor navigation capabilities. The system has not yet been tested in actual agricultural fields, so its performance under real-world conditions remains to be evaluated. Other aspects, such as more advanced navigation, adaptive spraying based on plant conditions, coordination between multiple robots, and remote monitoring, also represent possibilities beyond the current scope of the project.

Overall, the project provides a simulation-based demonstration of KhetiBot and highlights the potential of combining robotics, computer vision, and simulation technologies for agricultural applications.

REFERENCES

Baltazar, A. R., Santos, F. N. D., Moreira, A. P., Valente, A., & Cunha, J. B. (2021). Smarter robotic sprayer system for precision agriculture. Electronics, 10(17), 2061.

Jiang, A., & Ahamed, T. (2023). Navigation of an autonomous spraying robot for orchard operations using LiDAR for tree trunk detection. Sensors, 23(10), 4808.

Ribeiro, J. P. L., Gaspar, P. D., Soares, V. N. G. J., & Caldeira, J. M. L. P. (2022). Computational simulation of an agricultural robotic rover for weed control and fallen fruit Collection Algorithms for image detection and recognition and systems control, regulation, and command. Electronics, 11(5), 790.